

Projet 7 — Base de données NoSQL

Concevez et analysez une base de données NoSQL

Association NosCités • Paris & Lyon • Été 2024

Contexte du projet et importation des données

Qui ?

Association NosCités — surveille les plateformes de location courte durée (Airbnb, etc.)

Objectif

Mesurer l'impact des plateformes sur l'offre de logements à Paris et Lyon.

Problématique

Crash total de la base de données des locations à Paris dès la 1ère semaine.

Une sauvegarde a été retrouvée — son intégrité doit être vérifiée.

Données utilisées

Fichier : listings_Paris.csv | Source : scraping Airbnb juin 2024
95 885 documents importés dans MongoDB (collection : listings_paris)

Importation MongoDB

Outil : MongoDB Compass

Base : noscites

Collection : listings_paris

Total documents :

95 885

Partie 1 — Restaurer & comprendre les données

```
_id: ObjectId('6a1ea2e65781297a35db8429')
id: 80260
listing_url: "https://www.airbnb.com/rooms/80260"
scrape_id: 20240610195007
last_scraped: 2024-06-13T00:00:00.000+00:00
source: "previous scrape"
name: "Nice studio in Jourdain's village"
picture_url: "https://a0.muscache.com/pictures/716553/6c14f251_original.jpg"
host_id: 333548
host_url: "https://www.airbnb.com/users/show/333548"
host_name: "Charlotte"
host_since: 2011-01-03T00:00:00.000+00:00
host_location: "Paris, France"
host_about: "My name is Charlotte, I'm 26 and I study cinema production. I'll be go..."
host_response_time: "N/A"
host_response_rate: "N/A"
host_acceptance_rate: "N/A"
host_is_superhost: false
host_thumbnail_url: "https://a0.muscache.com/im/users/333548/profile_pic/1350320876/origina..."
host_picture_url: "https://a0.muscache.com/im/users/333548/profile_pic/1350320876/origina..."
host_neighbourhood: "Buttes-Chaumont - Belleville"
host_listings_count: 1
host_total_listings_count: 1
host_verifications: "['email', 'phone']"
host_has_profile_pic: true
host_identity_verified: true
neighbourhood_cleansed: "Ménilmontant"
latitude: 48.87131
longitude: 2.38848
property_type: "Entire rental unit"
room_type: "Entire home/apt"
accommodates: 3
bathrooms_text: "1 bath"
bedrooms: 1
amenities: Array (10)
  minimum_nights: 2
  maximum_nights: 730
  minimum_minimum_nights: 2
  maximum_minimum_nights: 2
  minimum_maximum_nights: 730
  maximum_maximum_nights: 730
```

*la description du 1^{er} document
avec le type des données*

```
minimum_nights_avg_ntm: 2
maximum_nights_avg_ntm: 730
has_availability: true
availability_30: 0
availability_60: 0
availability_90: 0
availability_365: 0
calendar_last_scraped: 2024-06-13T00:00:00.000+00:00
number_of_reviews: 206
number_of_reviews_ltm: 0
number_of_reviews_l30d: 0
first_review: 2011-04-18T00:00:00.000+00:00
last_review: 2021-10-03T00:00:00.000+00:00
review_scores_rating: 4.63
review_scores_accuracy: 4.61
review_scores_cleanliness: 4.75
review_scores_checkin: 4.85
review_scores_communication: 4.78
review_scores_location: 4.61
review_scores_value: 4.64
license: 7512005340473
instant_bookable: false
calculated_host_listings_count: 1
calculated_host_listings_count_entire_homes: 1
calculated_host_listings_count_private_rooms: 0
calculated_host_listings_count_shared_rooms: 0
reviews_per_month: 1.29
```

Partie 1 — Restaurer & comprendre les données

Catégorie	Exemples de champs	Type
Identifiants	_id, id, scrape_id, license	ObjectId, Integer, NumberLong
Informations générales	listing_url, source, name, picture_url	String
Informations hôte	host_id, host_name, host_since, host_is_superhost	Integer, String, Date, Boolean
Localisation	host_location, neighbourhood_cleansed, latitude, longitude	String, Float
Logement	property_type, room_type, accommodates, bedrooms, bathrooms_text	String, Integer
Équipements	amenities	Array
Règles de séjour	minimum_nights, maximum_nights	Integer
Disponibilités	has_availability, availability_30, availability_365	Boolean, Integer
Avis	number_of_reviews, first_review, review_scores_rating	Integer, Date, Float
Indicateurs calculés	reviews_per_month, calculated_host_listings_count	Float, Integer

la description du 1^{er} document avec le type des données

Partie 1 — Restaurer & comprendre les données

⚡ Avantages du NoSQL

- **Schéma flexible** → champs optionnels sans casser la structure
- **Données imbriquées** → amenities stocké en Array JSON
- **Scalabilité** → 95 885 docs, MongoDB monte en charge facilement
- **Hétérogénéité** → chaque annonce est unique, le modèle document s'adapte
- **SQL** → trop rigide : jointures multiples, colonnes vides, schéma figé

🔍 Premières requêtes

Total documents :
→ 95 885

Logements disponibles
(has_availability: true) :
→ 90 173

Partie 2 — Analyser l'intégrité des données

À compléter avec l'étape suivante :

- Les requêtes MongoDB et leurs résultats
- La démarche pour connecter MongoDB à un outil de Business Intelligence
- Captures d'écran du tableau de bord
- Captures d'écran du processus de connexion

Partie 2 — Analyser l'intégrité des données

■ Les requêtes MongoDB et leurs résultats

1) Combien d'annonces y a-t-il par type de location ?

```
db.listings_paris.aggregate([
  { $group: { _id: "$room_type", total: { $sum: 1 } } },
  { $sort: { total: -1 } }
])
```

Type de location	Nb annonces	%
Entire home/apt	85 733	89,4%
Private room	8 975	9,4%
Hotel room	776	0,8%
Shared room	401	0,4%

Paris est dominée à **89,4%** par des logements entiers loués sur Airbnb. C'est le cœur de la problématique de NosCités : ces appartements entiers sont retirés du marché locatif classique, ce qui réduit l'offre de logements pour les Parisiens.

Partie 2 — Analyser l'intégrité des données

*2) Quelles sont les 5 annonces de location avec le plus d'évaluations ?
Et combien d'évaluations ont-elles ?*

```
db.listings_paris.find(  
  {},  
  { name: 1, number_of_reviews: 1, _id: 0 }  
).sort({ number_of_reviews: -1 }).limit(5)
```

#	Nom de l'annonce	Nb évaluations
1	Sweet & cosy room next to Canal Saint Martin ❤️	3 067
2	Double/Twin Room, close to Opera and the Louvre with breakfast included	2 620
3	Bed in Dorm of 8 Beds "The Big One" in Paris	2 294
4	Comfortable bed in shared rooms of 8 in Paris 12e	2 105
5	Nice Room for 2 people	2 048

Les 5 annonces les plus évaluées sont toutes des chambres (Private room ou Shared room), pas des appartements entiers. Logique : plus abordables, elles attirent plus de voyageurs et génèrent donc plus d'avis.

Partie 2 — Analyser l'intégrité des données

3) Quel est le nombre total d'hôtes différents ?

```
db.listings_paris.distinct("host_id").length
```

Métrique	Résultat
Hôtes uniques	71 979
Total annonces	95 885
Ratio	~1,33 annonce/hôte

Avec 71 979 hôtes pour 95 885 annonces, le ratio est de 1,33 annonce par hôte en moyenne. Cela signifie que la majorité des hôtes ne possèdent qu'un seul logement

Partie 2 — Analyser l'intégrité des données

*4) Quel est le nombre de locations réservables instantanément ?
Cela représente quelle proportion des annonces ?*

// Étape 1

```
db.listings_paris.countDocuments({ instant_bookable: true })
```

// Étape 2 (résultat étape 1 = 22094)

```
(22094 / 95885 * 100).toFixed(2) + " %"
```

Métrique	Résultat
Locations instant bookable	22 094
Total annonces	95 885
Proportion	23,04%

Seulement 23,04% des annonces sont réservables instantanément. La majorité des hôtes parisiens préfèrent valider manuellement chaque réservation — signe d'une gestion plus personnelle et prudente du logement.

Partie 2 — Analyser l'intégrité des données

**5) Est-ce que des hôtes ont plus de 100 annonces sur les plateformes ?
Et si oui qui sont-ils ? Cela représente quel pourcentage des hôtes ?**

Étape 1 : Trouver les hôtes avec plus de 100 annonces :

```
db.listings_paris.aggregate([
  { $group: {
    _id: "$host_id",
    host_name: { $first: "$host_name" },
    nb_annonces: { $sum: 1 }
  }},
  { $match: { nb_annonces: { $gt: 100 } } },
  { $sort: { nb_annonces: -1 } }
])
```

#	Hôte	Nb annonces
1	Blueground	730
2	Veeve	497
3	Pierre De WeHost	426
...	...	...

Étape 2 : Compter combien ils sont :

```
db.listings_paris.aggregate([
  { $group: { _id: "$host_id", nb_annonces: { $sum: 1 } } },
  { $match: { nb_annonces: { $gt: 100 } } },
  { $count: "nb_hotes_plus_100" }
])

{
  nb_hotes_plus_100: 24
}
```

Étape 3 : Calculer leur pourcentage parmi tous les hôtes : (24 résultats étape 2)

$(24 / 71979 * 100).toFixed(2) + " \%"$

< 0.03 %

Seulement 23,04% des annonces sont réservables instantanément. La majorité des hôtes parisiens préfèrent valider manuellement chaque réservation, c'est signe d'une gestion plus personnelle et prudente du logement.

Partie 2 — Analyser l'intégrité des données

*5) Est-ce que des hôtes ont plus de 100 annonces sur les plateformes ?
Et si oui qui sont-ils ? Cela représente quel pourcentage des hôtes ?*

Métrique	Résultat
Hôtes avec +100 annonces	24
Total hôtes	71 979
Proportion	0,03%

Seulement 24 hôtes sur 71 979 possèdent plus de 100 annonces, soit une infime minorité de 0,03%. Pourtant, ces acteurs ultra-professionnels gèrent une part significative du parc locatif Airbnb parisien. Ce sont souvent des sociétés de gestion locative qui industrialisent la location courte durée, accentuant la pression sur le marché immobilier parisien.

Partie 2 — Analyser l'intégrité des données

6) Combien y a-t-il de super hôtes différents ? Cela représente quel pourcentage des hôtes ?

// Étape 1 — Nombre de super hôtes uniques

```
db.listings_paris.distinct("host_id", { host_is_superhost: true }).length
```

// Étape 2 — Calculer le pourcentage (10027 = résultat de l'étape 2)

```
(10027 / 71979 * 100).toFixed(2) + " %"
```

Métrique	Résultat
Super hôtes uniques	10 027
Total hôtes (req. 3)	71 979
Proportion	13,93%

Avec seulement 13,93% de super hôtes, le label reste sélectif et difficile à obtenir. Il récompense les hôtes qui maintiennent un taux de réponse élevé, peu d'annulations et d'excellentes évaluations de la part des voyageurs. Cette faible proportion montre que la grande majorité des hôtes parisiens gèrent leur logement de façon occasionnelle ou peu optimisée, loin des standards professionnels qu'Airbnb met en avant.

Partie 2 — Analyser l'intégrité des données (*avec Polars*)

1) Calculer le taux de réservation moyen par mois par type de logement

```
req1 = (  
    df.with_columns([  
        ((365 - pl.col("availability_365")) / 365 * 100).alias("taux_reservation"),  
        pl.col("last_scraped").dt.month().alias("mois")  
    ])  
    .group_by(["mois", "room_type"])  
    .agg(pl.col("taux_reservation").mean().round(2).alias("taux_moyen"))  
    .sort(["mois", "room_type"])  
)
```

Mois	Type de logement	Taux de réservation moyen
Juin	Private room	68,80%
Juin	Entire home/apt	64,90%
Juin	Shared room	63,33%
Juin	Hotel room	52,38%

Les **chambres privées** affichent le taux de réservation le plus élevé à **68,8%**, devant les appartements entiers à **64,9%**. Les hôtels Airbnb sont les moins réservés avec seulement **52,38%** c'est logique, ils font face à une forte concurrence des hôtels traditionnels parisiens.

Partie 2 — Analyser l'intégrité des données (*avec Polars*)

2) Calculer la médiane des nombre d'avis pour tous les logements

```
req2 = df.select(  
    pl.col("number_of_reviews").median().alias("mediane_avis")  
)
```

Métrique	Résultat
Médiane des avis	3 avis

La médiane est de seulement **3 avis**, ce qui veut dire que **la moitié des logements parisiens sur Airbnb ont 3 avis ou moins**. C'est très faible ! Cela révèle qu'une grande majorité des annonces sont **peu actives ou récentes**, tandis qu'une minorité de logements très populaires concentrent des centaines d'avis (comme le top 5 qu'on a vus dans la requête 2 de l'étape 1 avec plus de 2 000 avis). C'est une distribution très **asymétrique**.

Partie 2 — Analyser l'intégrité des données (*avec Polars*)

3) Calculer la médiane des nombre d'avis par catégorie d'hôte

```
req3 = (  
    df.group_by("host_is_superhost")  
    .agg(pl.col("number_of_reviews").median().alias("mediane_avis"))  
    .sort("host_is_superhost")  
)
```

Catégorie d'hôte	Médiane des avis
Super hôte 	24 avis
Non super hôte 	2 avis
Non renseigné (null)	12,5 avis

La différence est **énorme**, les super hôtes ont une médiane de **24 avis** contre seulement **2 avis** pour les non super hôtes. C'est 12 fois plus ! Cela confirme que le label "super hôte" est directement lié à une **activité intense et régulière** sur la plateforme. Les super hôtes sont des hôtes expérimentés qui accueillent beaucoup plus de voyageurs, ce qui génère mécaniquement plus d'avis.

Partie 2 — Analyser l'intégrité des données (*avec Polars*)

4) Calculer la densité de logements par quartier de Paris

```
req4 = (  
    df.group_by("neighbourhood_cleansed")  
    .agg(pl.len().alias("nb_logements"))  
    .sort("nb_logements", descending=True)  
)
```

#	Quartier	Nb logements
1	Buttes-Montmartre	10 555
2	Popincourt	8 430
3	Vaugirard	7 802
4	Batignolles-Monceau	6 857
5	Entrepôt	6 558
...	...	...
16	Élysée	2 898
17	Hôtel-de-Ville	2 821
18	Palais-Bourbon	2 740
19	Luxembourg	2 701
20	Louvre	2 026

Buttes-Montmartre domine largement avec **10 555 logements**, soit 5x plus que le quartier le moins dense (Louvre). Les quartiers les plus denses sont des **zones résidentielles populaires** comme Montmartre, Popincourt ou Vaugirard, pas comme les quartiers touristiques du centre. Paradoxalement, les quartiers ultra-touristiques comme le **Louvre** ou le **Luxembourg** ont peu d'annonces, probablement à cause des prix immobiliers très élevés qui limitent l'offre.

Partie 2 — Analyser l'intégrité des données (*avec Polars*)

5) Identifier les quartiers avec le plus fort taux de réservation par mois

```
req5 = (  
    df.with_columns([  
        ((365 - pl.col("availability_365")) / 365 * 100).alias("taux_reservation")  
    ])  
    .group_by("neighbourhood_cleansed")  
    .agg(pl.col("taux_reservation").mean().round(2).alias("taux_moyen"))  
    .sort("taux_moyen", descending=True)  
)
```

Ménilmontant arrive en tête avec **71,08%** de taux de réservation, suivi des quartiers populaires du nord-est parisien. C'est une tendance claire : les **quartiers résidentiels et populaires** (Ménilmontant, Buttes-Chaumont, Montmartre) sont plus réservés que les quartiers **huppés et touristiques** comme l'Élysée ou Passy. Cela s'explique par des prix plus abordables qui attirent davantage de voyageurs à la recherche de bonnes affaires.

#	Quartier	Taux de réservation
1	Ménilmontant	71,08%
2	Buttes-Chaumont	69,73%
3	Buttes-Montmartre	69,25%
4	Entrepôt	68,99%
5	Popincourt	68,87%
...	...	...
16	Luxembourg	60,24%
17	Louvre	57,97%
18	Bourse	57,49%
19	Passy	56,62%
20	Élysée	53,68%

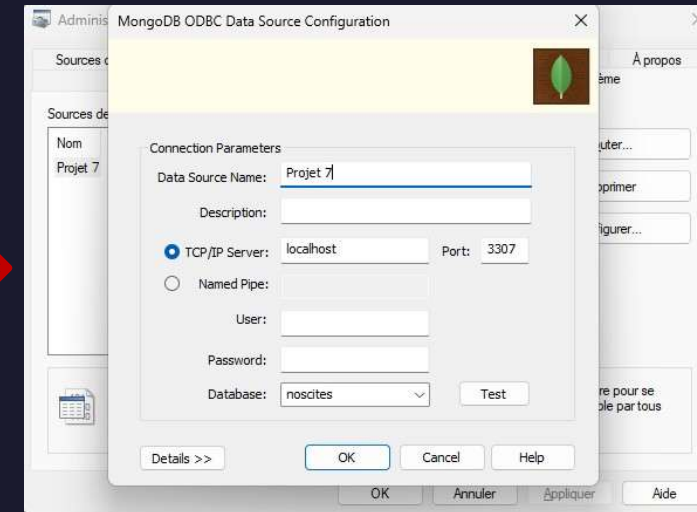
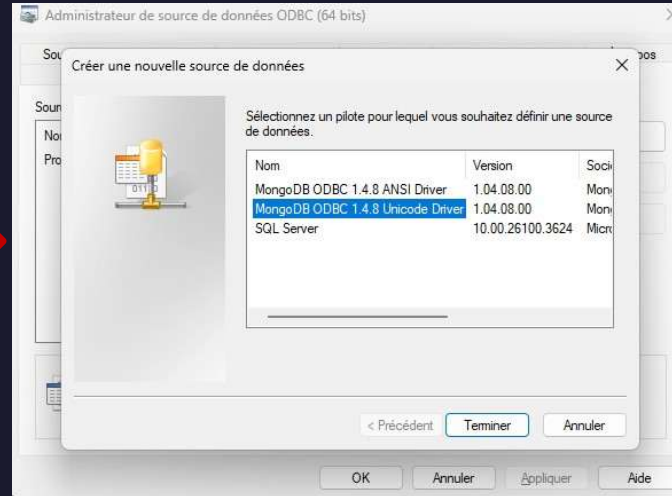
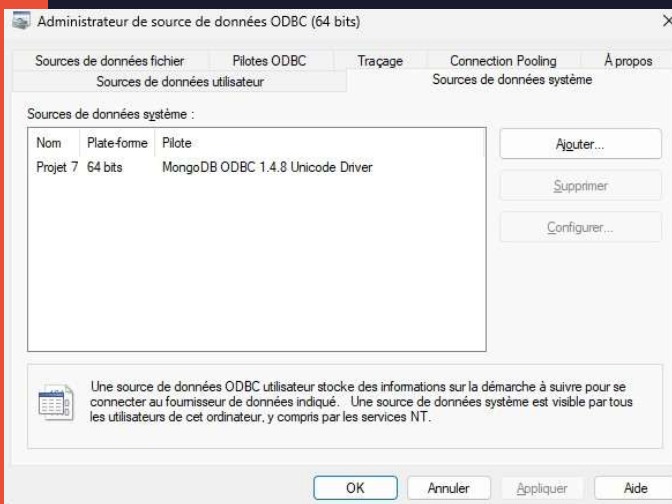
Partie 2 — Analyser l'intégrité des données

■ La démarche pour connecter MongoDB à un outil de Business Intelligence

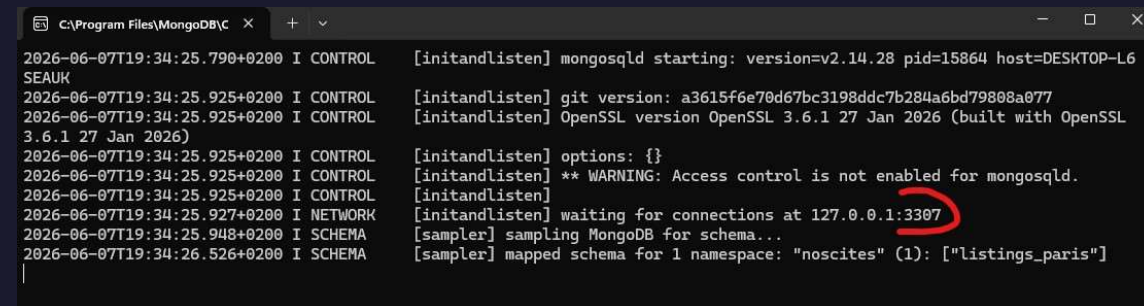
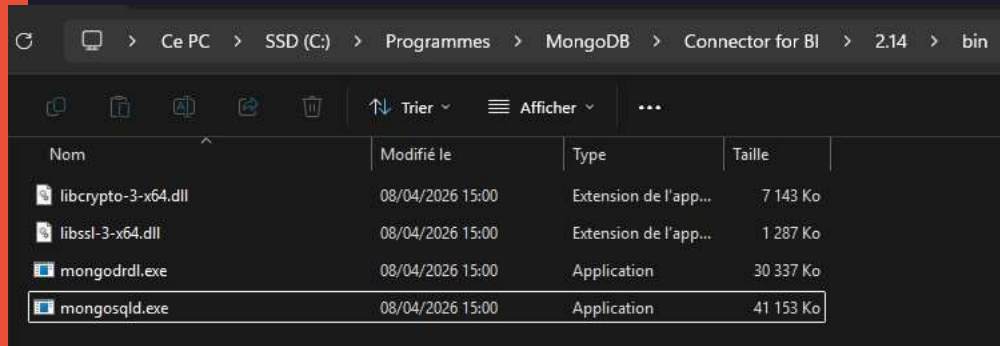
1) Télécharger mongodb-connector-odbc

2) Cliquez sur ajouter et "Unicode"

3) Configurer la source



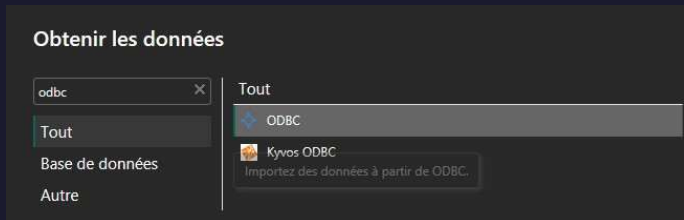
3.1) Configurer la source : télécharger : MongoDB Connector for BI



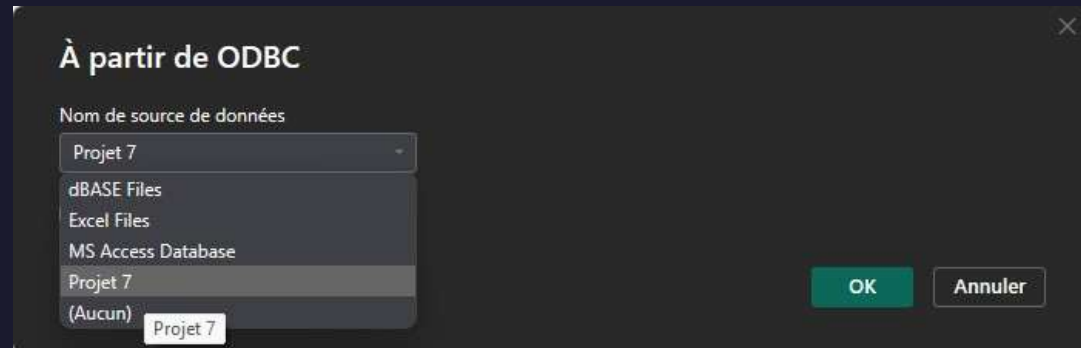
Partie 2 — Analyser l'intégrité des données

- La démarche pour connecter MongoDB à un outil de Business Intelligence

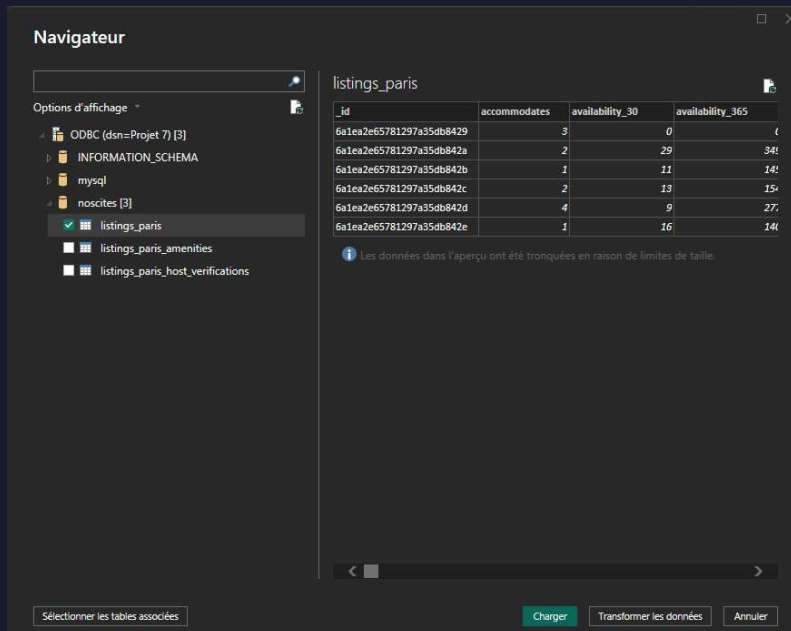
4) Ouvre Power BI → obtenir des données



5) Sélectionne la source que l'on avait créée



6) Sélectionne la bonne base de données



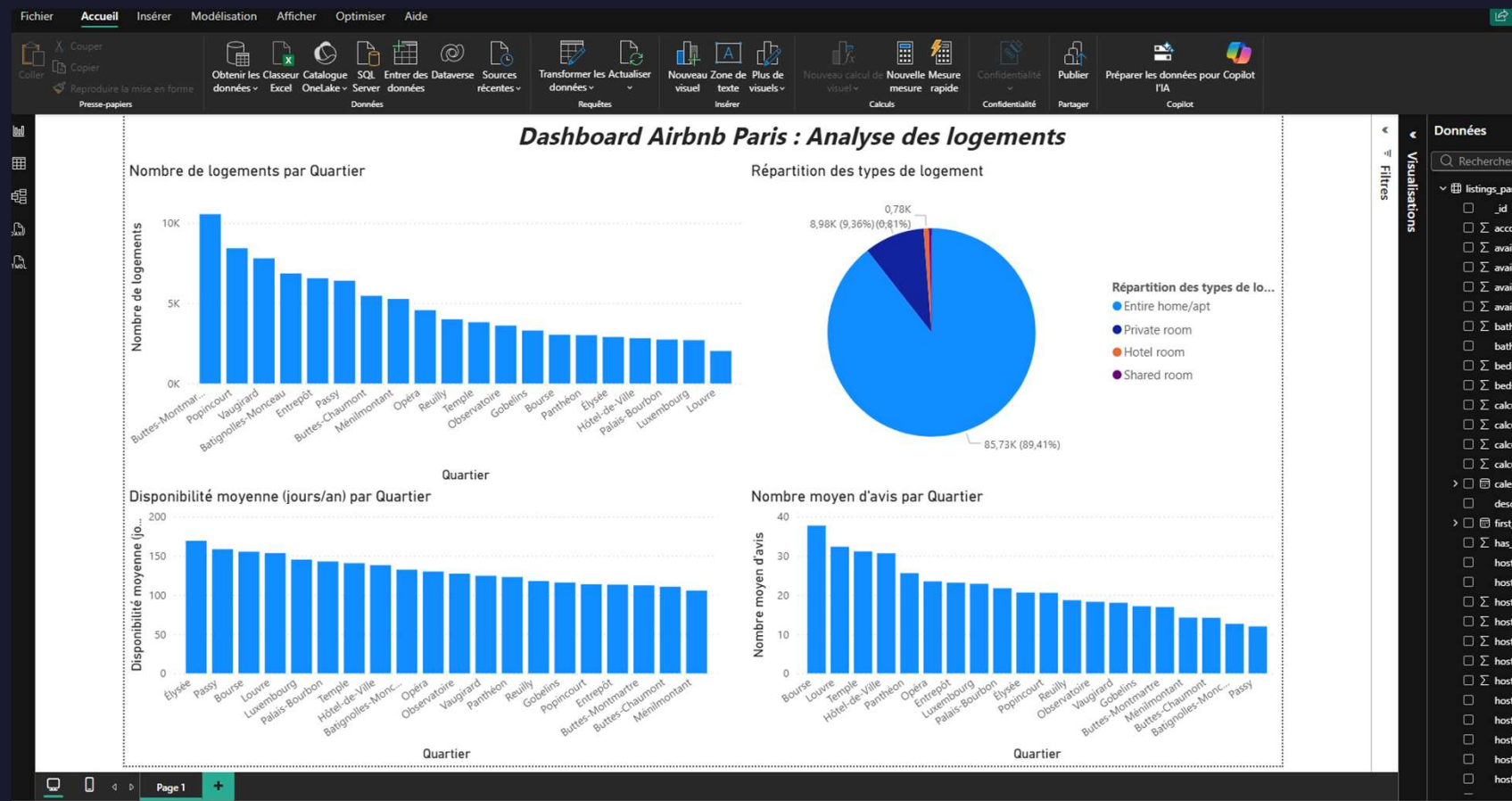
Partie 2 — Analyser l'intégrité des données

■ La démarche pour connecter MongoDB à un outil de Business Intelligence

The screenshot displays a BI tool interface with a dark theme. The top menu bar includes 'Fichier', 'Accueil', 'Aide', and 'Outils de table'. Below the menu is a toolbar with icons for 'Nom' (set to 'listings_paris'), 'Structure', 'Relations', 'Calculs', and 'Calendriers'. The main area shows a data table with columns: _id, accommodates, availability_30, availability_365, availability_60, availability_90, bathrooms, bathrooms_text, bedrooms, beds, calculated_host_listings_count, calculated_host_listings_count_entire_homes, and calculated. The table contains 32 rows of data. On the right, a 'Données' pane shows a search bar and a list of fields from the 'listings_paris' source, including _id, accommodates, availability_30, availability_365, availability_60, availability_90, bathrooms, bathrooms_text, bedrooms, beds, calculated_host_listings_count, calculated_host_listings_count_entire_homes, calculated, calendar_last_scraped, description, first_review, has_availability, host_about, host_acceptance_rate, and host_availability.

_id	accommodates	availability_30	availability_365	availability_60	availability_90	bathrooms	bathrooms_text	bedrooms	beds	calculated_host_listings_count	calculated_host_listings_count_entire_homes	calculated
6a1ea2e65781297a35db89b5	2	28	88	58	88	1	1 bath	1	1	1	1	1
6a1ea2e65781297a35db8e4b	3	30	189	60	80	1	1 bath	1	1	1	1	1
6a1ea2e75781297a35db98cf	2	7	55	37	55	1	1 bath	1	0	1	1	1
6a1ea2e85781297a35dbaa0a	4	0	101	18	33	1	1 bath	1	2	1	1	1
6a1ea2e85781297a35dbadd0	2	0	32	20	32	1	1 bath	1	1	1	1	1
6a1ea2e95781297a35dbaff6	3	12	347	42	72	1	1 bath	1	2	1	1	1
6a1ea2ea5781297a35dbbe9b	3	27	267	57	87	1	1 bath	1	1	1	1	1
6a1ea2eb5781297a35dbc5fe	2	0	330	25	55	1	1 bath	1	1	1	1	1
6a1ea2ec5781297a35dbd004	2	0	50	21	50	1	1 bath	1	1	1	1	1
6a1ea2ec5781297a35dbd579	4	0	35	16	35	1	1 bath	1	2	1	1	1
6a1ea2ed5781297a35dbdf39	4	29	269	59	89	1	1 bath	1	1	1	1	1
6a1ea2ee5781297a35dbe576	2	10	189	40	70	1	1 bath	1	1	1	1	1
6a1ea2ee5781297a35dbe59f	4	7	227	37	47	1	1 bath	1	1	1	1	1
6a1ea2f05781297a35dbfbad	4	0	140	18	25	1	1 bath	1	2	1	1	1
6a1ea2f05781297a35dbfc63	3	30	270	60	90	1	1 bath	1	1	1	1	1
6a1ea2f05781297a35dbfe4f	4	29	269	59	89	1	1 bath	1	3	1	1	1
6a1ea2f05781297a35dc0058	4	27	362	57	87	1	1 bath	1	2	1	1	1
6a1ea2f05781297a35dc008e	2	0	45	21	21	1	1 bath	1	1	1	1	1
6a1ea2f15781297a35dc0a05	2	29	269	59	89	1	1 bath	1	1	1	1	1
6a1ea2f15781297a35dc0d2e	2	3	77	33	50	1	1 bath	1	0	1	1	1
6a1ea2f25781297a35dc18ce	4	30	270	60	90	1	1 bath	1	2	1	1	1
6a1ea2f25781297a35dc1b4f	4	24	359	54	84	1	1 bath	1	2	1	1	1
6a1ea2f25781297a35dc205b	4	11	346	41	71	1	1 bath	1	3	1	1	1
6a1ea2f25781297a35dc20f7	2	30	365	60	90	1	1 bath	1	1	1	1	1
6a1ea2f25781297a35dc2149	2	10	345	40	70	1	1 bath	1	1	1	1	1
6a1ea2f25781297a35dc2532	2	0	16	16	16	1	1 bath	1	1	1	1	1

■ Captures d'écran du tableau de bord



Partie 3 — Pérenniser la base de données

Étape 1 : Import des données Paris + Lyon

Les données Lyon ont été téléchargées puis taguées avec un champ city: Lyon via un script Python avant l'import. Les données Paris existantes ont été taguées city: Paris via un updateMany. Les deux villes cohabitent dans la même collection listings_paris.

```
test> use noscites
switched to db noscites
noscites> db.listings_paris.aggregate([
|   { $group: { _id: "$city", count: { $sum: 1 } } }
| ])
[ { _id: 'Lyon', count: 9973 }, { _id: 'Paris', count: 95885 } ]
```

```
db.listings_paris.updateMany(
  { city: { $exists: false } },
  { $set: { city: "Paris" } }
)
```

```
df = pd.read_csv("listings_Lyon+.csv", low_memory=False)
df["city"] = "Lyon"
df.to_csv("listings_lyon_tagged.csv", index=False, encoding="utf-8")
```

Partie 3 — Pérenniser la base de données

Étape 2 : Réplication des données avec ReplicaSet

La stratégie de réplication vise à protéger les données contre les pannes matérielles. Si le Primary tombe, le Secondary prend automatiquement le relais grâce au vote de l'Arbitre. Aucune donnée n'est perdue et le service reste disponible.

PRIMARY

```
members: [
  {
    _id: 0,
    name: 'localhost:27020',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 116,
    optime: { ts: Timestamp({ t: 1781018665, i: 1 }),
    optimeDate: ISODate('2026-06-09T15:24:25.000Z'),
```

SECONDARY

```
{
  _id: 1,
  name: 'localhost:27021',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 70,
  optime: { ts: Timestamp({ t: 1781018665, i: 1 }),
  optimeDurable: { ts: Timestamp({ t: 1781018665, i
```

ARBITER

```
{
  _id: 2,
  name: 'localhost:27022',
  health: 1,
  state: 7,
  stateStr: 'ARBITER',
  uptime: 70,
  lastHeartbeat: ISODate('2026-06-09T15:24:25.000Z'),
  lastHeartbeatRecv: ISODate('2026-06-09T15:24:25.000Z'),
```

Vérification fonctionnement :

Connection PRIMARY (port 27020) :

```
db.test_replication.insertOne({ message: "test replication", city: "Paris" })
```

Connection SECONDARY (port 27021) :

```
db.test_replication.find()
```

```
rs0 [direct: secondary] noscites> db.test_replication.find()
[
  {
    _id: ObjectId('6a28309193f6995323abc114'),
    message: 'test replication',
    city: 'Paris'
  }
]
```

Un ReplicaSet de 3 nœuds a été créé en local sur 3 ports différents pour simuler plusieurs serveurs physiques. Le Primary reçoit toutes les écritures, le Secondary réplique les données en temps réel, l'Arbitre gère les votes d'élection en cas de panne.

Partie 3 — Pérenniser la base de données

Étape 3 : Distribution des données avec le Sharding

Un cluster de sharding a été mis en place en local avec 2 shards simulant les serveurs Paris et Lyon. Le Mongos joue le rôle de routeur unique : chaque requête est automatiquement dirigée vers le bon shard selon la ville, ce qui optimise les performances des requêtes locales.

Composant	Rôle	Port
Config Server	Stocke la carte du cluster	27030
Mongos	Routeur des requêtes	27050
shardRS1	Données Paris	27040
shardRS2	Données Lyon	27041

```
shards
[
  {
    _id: 'shardRS1',
    host: 'shardRS1/localhost:27040',
    state: 1,
    topologyTime: Timestamp({ t: 1781030930, i: 9 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'shardRS2',
    host: 'shardRS2/localhost:27041',
    state: 1,
    topologyTime: Timestamp({ t: 1781030930, i: 66 }),
    replSetConfigVersion: Long('-1')
  }
]
```

Les 2 shards sont bien actifs (state: 1)

Partie 3 — Pérenniser la base de données

Étape 3 : Vérification de la collection shardée

Vérification que la collection listings_paris est bien distribuée sur le cluster de sharding

```
[direct: mongos] noscites> db.listings_paris.getShardDistribution()
Shard shardRS1 at shardRS1/localhost:27040
{
  data: '0B',
  docs: 0,
  chunks: 1,
  'estimated data per chunk': '0B',
  'estimated docs per chunk': 0
}
---
Totals
{
  data: '0B',
  docs: 0,
  chunks: 1,
  'Shard shardRS1': [ '0 % data', '0 % docs in cluster', '0B avg obj size on shard' ]
}
[direct: mongos] noscites> |
```

La collection listings_paris est shardée avec la clé { city: 1 } et le balancing est activé

```
collections: {
  'noscites.listings_paris': {
    shardKey: { city: 1 },
    unique: false,
    balancing: true,
    allowMigrations: true,
    chunkMetadata: [ { shard: 'shardRS1', nChunks: 1 } ],
    chunks: [
      { min: { city: MinKey() }, max: { city: MaxKey() }, 'on shard': 'shardRS1', 'last modified': Timestamp({ t: 1, i: 0 }) }
    ],
    tags: []
  }
}
```

Confirmation que la collection est bien distribuée sur le cluster

La clé de sharding choisie est le champ city. Ce choix est logique car les données sont naturellement partitionnées par ville : une requête Lyon va directement sur shardRS2 sans scanner les données Paris, et inversement

Partie 3 — Pérenniser la base de données

Étape 1, 2 et 3 : Processus de réplication et distribution des données sous forme de tableau

#	Étape	Objectif	Commandes exécutées	Outils	Résultat
1	Import des données Paris + Lyon	Centraliser toutes les données dans une seule collection avec distinction par ville	Script Python pour tagger city: "Lyon" + mongoimport pour Lyon : updateMany pour tagger city: "Paris"	Python, mongoimport, MongoDB CLI	95 885 docs Paris + 9 973 docs Lyon dans la collection listings_paris distingués par le champ city
2	Création des dossiers	Préparer l'espace de stockage pour chaque instance	mkdir C:\data\rs0 mkdir C:\data\rs1 mkdir C:\data\rs2	Windows CMD	3 dossiers créés pour le ReplicaSet
3	Lancement du ReplicaSet (3 instances)	Simuler 3 serveurs en local pour la réplication	mongod --replSet rs0 --port 27020 --dbpath C:\data\rs0 mongod --replSet rs0 --port 27021 --dbpath C:\data\rs1 mongod --replSet rs0 --port 27022 --dbpath C:\data\rs2	MongoDB CLI	3 instances actives sur ports 27020, 27021, 27022
4	Initialisation du ReplicaSet	Définir les rôles PRIMARY / SECONDARY / ARBITRE	rs.initiate({ _id: "rs0", members: [...] })	mongosh	ok: 1 — ReplicaSet rs0 opérationnel
5	Vérification de la réplication	Confirmer que les données se répliquent bien entre les nœuds	db.test_replication.insertOne(...) sur Primary puis db.test_replication.find() sur Secondary	mongosh	Document visible sur le Secondary → réplication fonctionnelle 
6	Création des dossiers Sharding	Préparer l'espace de stockage pour le cluster	mkdir C:\data\configdb mkdir C:\data\shard1 mkdir C:\data\shard2	Windows CMD	3 dossiers créés pour le cluster
7	Lancement du Config Server	Stocker la carte du cluster (métadonnées)	mongod --configsvr --replSet configRS --port 27030 --dbpath C:\data\configdb	MongoDB CLI	Config Server actif sur port 27030
8	Initialisation du Config Server	Rendre le Config Server opérationnel	rs.initiate() sur port 27030	mongosh	ok: 1 
9	Lancement des 2 Shards	Simuler 2 serveurs de stockage (Paris + Lyon)	mongod --shardsvr --replSet shardRS1 --port 27040 mongod --shardsvr --replSet shardRS2 --port 27041	MongoDB CLI	2 shards actifs sur ports 27040 et 27041
10	Initialisation des 2 Shards	Rendre les shards opérationnels	rs.initiate() sur port 27040 et 27041	mongosh	ok: 1 pour les 2 
11	Lancement du Mongos (routeur)	Créer le point d'entrée unique du cluster	mongos --configdb configRS/localhost:27030 --port 27050	MongoDB CLI	Routeur actif sur port 27050
12	Ajout des Shards au cluster	Enregistrer les 2 shards dans le cluster	sh.addShard("shardRS1/localhost:27040") sh.addShard("shardRS2/localhost:27041")	mongosh (port 27050)	shardAdded: shardRS1 et shardAdded: shardRS2 
13	Activation du Sharding sur la BDD	Autoriser la distribution de la base noscites	sh.enableSharding("noscites")	mongosh	ok: 1 
14	Sharding de la collection	Distribuer listings_paris par ville	sh.shardCollection("noscites.listings_paris", { "city": 1 })	mongosh	collectionsharded: noscites.listings_paris 
15	Vérification finale	Confirmer la distribution sur les shards	sh.status() + db.listings_paris.getShardDistribution()	mongosh	2 shards actifs, collection shardée par city, balancer activé 

Partie 3 — Pérenniser la base de données

Schéma d'architecture de la base de données incluant la connexion à un outil de business intelligence

